zkLogin contribution guidelines

Summary

In this document we want to support the developer in the following ways:

- Contribution guidelines
- Understand the protocol and its implementation
- Add new endpoints to the server
- Improve and add tests

Contribution Guide

This project combines:

- a React/Vite frontend in `frontend/`,
- a Node backend in `backend/`,
- Circom circuits in `backend/circuits/` ,
- Aiken validator code in `backend/validators/`.

Before You Start

- Read the main `README.md` for setup instructions and running the project locally.
- Keep changes scoped to one concern when possible.
- If you change the frontend flow or backend API, update the corresponding documentation too.

Development Expectations

- Prefer small, reviewable pull requests.
- Preserve the existing project structure and naming conventions.
- Avoid committing secrets, private keys, or personal `.env` values.
- If you add a new endpoint, also add or update backend tests.

Pull Request Checklist

Before opening a PR, make sure:

- the change is scoped and explained clearly,
- affected tests were added or updated,
- local verification was done for the affected area,
- related docs were updated,
- no generated noise or secret material was accidentally committed.

zkLogin protocol

- Protocol explained: https://github.com/eryxcoop/zklogin-aiken/blob/main/report/report.pdf
- Original paper: https://arxiv.org/pdf/2401.11735

Backend Server Guide

This document explains how `src/server.ts` is structured, how requests move through the server, and how to add a new endpoint without breaking the current pattern.

Purpose

The backend server is a small HTTP entrypoint built directly on Node's built-in `http` module. It does not use Express, Fastify, or another web framework.

`server.ts` has one main responsibility: route incoming HTTP requests to the correct endpoint handler module.

The endpoint handlers contain the actual application logic, and those handlers in turn call the domain-specific deployment code under `deployment/`.

High-Level Structure

The current server has four layers:

1. HTTP server bootstrap in `src/server.ts`
2. Endpoint handler modules in `src/handleDeriveAddressEndpoint.ts`, `src/handleGenerateProofEndpoint.ts`, `src/handleFundAddressEndpoint.ts`, and `src/handleTransferFundsEndpoint.ts`
3. Deployment and blockchain logic under `deployment/`

4. Tests under the `src/` directory, such as `src/deriveAddressEndpoint.test.ts`, `src/generateProofEndpoint.test.ts`, and `src/fundAddressEndpoint.test.ts`

This split is intentional:

- `server.ts` handles transport concerns.
- Endpoint handlers translate HTTP-shaped input into domain calls.
- Deployment modules perform the heavy work.

Request Lifecycle

Every request follows this path:

1. Node receives the request through `http.createServer(requestListener)`.
2. `requestListener` sets CORS headers.
3. If the request method is `OPTIONS`, the server returns immediately for preflight handling.
4. The request URL is parsed with `urlFromRequest(...)`.
5. `server.ts` matches the pair `(HTTP method, pathname)` against a route.
6. For `POST` routes, the raw Node request is converted into a simpler request object by `convertNodeServerRequestToRequest(...)`. This conversion is important because it simplifies the request object and allows the endpoint to be tested with a mock.
7. The selected endpoint handler is called.
8. The handler returns an object shaped like:

```
None
{
  status: number,
  body: Record<string, unknown>
}
```

9. `server.ts` writes the HTTP status and serializes the response body as JSON.

Server configuration

The host and port are currently hardcoded:

- `host = "localhost"`
- `port = 8000`

If you change the port here, you must also update the frontend constants in `frontend/src-frontend/constant.ts`.

`convertNodeServerRequestToRequest(nodeServerRequest)`

Note: it is important to wrap the node request so we can easily emulate requests in tests

Reads and parses the JSON body of a `POST` request and returns a simplified request object:

```
None
{
  url,
  method,
  headers,
  body
}
```

This helper function is what lets the endpoint handlers operate on a lightweight request structure instead of the raw Node stream API.

Important limitation:

- it assumes the request body is JSON,
- it calls `JSON.parse(data)`,
- an empty or invalid JSON body will throw an Error.

That is acceptable for the current endpoints, but if you add routes with empty bodies or form data, this helper will need to be extended.

How To Add A New Endpoint

Suppose you want to add `POST /healthCheckExample` or another route with real business logic. The safest way is:

1. Create a dedicated handler

Add a new file under `src/`, for example `src/handleHealthCheckExampleEndpoint.ts`.

Recommended shape:

None

```
export async function handleHealthCheckExampleEndpoint(request) {
  try {
    const someField = request.body["someField"];

    const result = await someDomainFunction(someField);

    return {
      status: 200,
      body: {
        message: "Health check example succeeded.",
        result,
      },
    };
  } catch (error) {
    return {
      status: 500,
      body: {
        message: "Health check example failed.",
        error: error.message,
      },
    };
  }
}
```

Keep transport logic minimal here. If the endpoint does meaningful business work, move that work into a separate module under `deployment/` or another domain-focused folder.

2. Import the handler in `server.ts`

Add an import near the top of `src/server.ts`:

None

```
import {handleHealthCheckExampleEndpoint} from
"./handleHealthCheckExampleEndpoint.ts";
```

3. Add a pathname constant

Near the existing route constants:

```
None
const HEALTH_CHECK_EXAMPLE_PATHNAME = "/healthCheckExample";
```

4. Register the route in `requestListener`

Add a new branch following the existing pattern:

```
None
} else if (nodeServerRequest.method === "POST" && url.pathname
=== HEALTH_CHECK_EXAMPLE_PATHNAME) {
    process.stdout.write("Received request to health check
example... ");
    const request = await
convertNodeServerRequestToRequest(nodeServerRequest);
    const response = await
handleHealthCheckExampleEndpoint(request);
    nodeServerResponse.writeHead(response.status,
{"Content-Type": "application/json"});
    nodeServerResponse.end(JSON.stringify(response.body));
    process.stdout.write("done.\n");
```

If the endpoint is `GET` and only needs query params, follow the `/deriveAddress` pattern instead.

5. Add tests for the handler

Create a test file under `src/`, for example `src/healthCheckExampleEndpoint.test.ts`.

Follow the existing test style:

- import the handler directly,
- call it with a plain object representing the request,
- assert on `status` and `body`,
- inject fake logic when the real dependency is expensive or external.

The helper assertions are in `tests_common_code/responseAssertions.ts`.

6. Update the frontend or clients

If the endpoint is called from the frontend, also update:

- `frontend/src-frontend/constant.ts` if the route should be configurable
- the relevant frontend code in `frontend/src-frontend/App.tsx`

Contribution Guidelines Summary

When adding a new endpoint, keep these rules:

- Keep `server.ts` thin. It should route, not contain business logic.
- Put reusable or complex behavior outside the handler.
- Return consistent JSON objects with `status` and a descriptive `body`.
- Catch errors inside the handler and convert them into explicit response bodies.
- Add a focused test file for each new handler.

Common Pitfalls

- Adding a `POST` endpoint that does not send JSON, which will fail in `convertNodeServerRequestToRequest(...)`.
- Putting endpoint logic directly in `server.ts`, which makes testing harder.
- Returning a raw value from a handler instead of `{ status, body }`.
- Changing a route path in the backend without updating the frontend caller.

Suggested Future Improvements

The current server is intentionally simple, but if it grows, these refactors would help:

- extract route dispatch into a route table instead of a long `if/else if` chain,
- centralize repeated request-to-handler and handler-to-response boilerplate,
- add request validation before calling handlers,
- move host, port, and allowed origin into environment variables,
- standardize error response shapes across all endpoints.

For the current size of the project, the existing structure is still manageable, but new endpoints should continue following the current modular handler pattern.

Backend Testing Guide

This backend already has a real test suite. It uses Node's built-in test runner through the `test` script in `package.json`:

```
None
npm run test
```

The suite is organized by concern rather than by a single `tests/` folder.

How Tests Are Organized

There are four main test areas in this backend.

1. Endpoint handler tests in `src/`

These files test the HTTP-facing handler modules directly, without starting the real server:

- `src/deriveAddressEndpoint.test.ts`
- `src/generateProofEndpoint.test.ts`
- `src/fundAddressEndpoint.test.ts`

Pattern:

- construct a minimal request-like object,
- call the handler directly,
- assert on the returned `{ status, body }` object.

Examples:

- query-string based endpoint testing in `src/deriveAddressEndpoint.test.ts`
- injected fake proof generation in `src/generateProofEndpoint.test.ts`
- injected success and failure behavior in `src/fundAddressEndpoint.test.ts`

This layer is the fastest way to validate request parsing and response shapes.

2. Deployment tests in `deployment/`

These files test the backend's domain and proof-generation logic more directly:

- `deployment/deriveAddress.test.ts`
- `deployment/generateProof.test.ts`

These tests sit below the HTTP layer and focus on the actual functions used by the endpoints.

Current pattern:

- `deriveAddress.test.ts` checks a deterministic prefix on the derived address.
- `generateProof.test.ts` prepares a temporary output directory, runs proof generation, and validates the output proof file format.

3. Circuit tests in `circuit_tests/`

`circuit_tests/zkLogin.test.ts` covers the Circom circuits using `circom_tester`.

This file validates:

- JWT signature verification circuits,
- bit and limb conversion helper circuits,
- the main zkLogin circuit without signature verification,
- the full zkLogin circuit with signature verification.

These are the lowest-level tests in the repo and are the closest to the Zero Knowledge circuits.

4. Shared test helpers in `tests_common_code/`

This folder contains reusable test support code:

- `tests_common_code/responseAssertions.ts`: small assertion helpers for endpoint responses
- `tests_common_code/testDataForACompleteFlowOfZkLogin.ts`: canonical JWT, circuit inputs, and conversion helpers reused across tests
- `tests_common_code/writeProofInFile.ts`: writes a deterministic example proof for tests that should avoid real proof generation

This shared layer keeps long fixtures and repeated assertions out of individual test files.

Current Coverage Shape

The current suite is broad but not uniform.

Covered now:

- derive-address endpoint behavior
- generate-proof endpoint behavior
- fund-address endpoint behavior
- derive-address deployment logic

- generate-proof deployment logic
- circuit-level zkLogin and helper circuit behavior

Not currently covered by a committed test file:

- `src/handleTransferFundsEndpoint.ts`
- full server routing in `src/server.ts`

That gap matters when you change transfer request parsing, response shape, or route registration.

Test Style Used In This Project

The existing suite follows a consistent style:

- use `describe` and `it` from `node:test`,
- use `node:assert/strict` or shared assertion helpers,
- prefer direct function calls over booting the full server,
- inject fake side effects when the real implementation is slow or external,
- keep reusable fixtures in `tests_common_code/`.

For example, the proof tests switch between a real proof run and a stubbed proof writer based on `REAL_PROOF`.

How To Add A New Backend Test

Pick the test location based on what you are changing.

Add a `src/` test when you change an endpoint handler

Use this for:

- request parsing,
- query param handling,
- JSON body handling,
- returned HTTP status and body shape.

Recommended file naming:

- `src/<name>Endpoint.test.ts`

Example:

- if you add a new endpoint handler `src/handleHealthCheckEndpoint.ts`, add `src/healthCheckEndpoint.test.ts`

Recommended structure:

1. import the handler directly,
2. build the smallest request object needed,
3. inject a fake dependency if the real one touches the network, filesystem, or blockchain,
4. assert on `response.status`,
5. assert on the important fields in `response.body`.

Add a `deployment/` test when you change domain logic

Use this for:

- address derivation logic,
- proof generation orchestration,
- filesystem output contracts,
- blockchain transaction preparation logic.

Recommended file naming:

- `deployment/<module>.test.ts`

These tests should stay one layer below HTTP and verify the business logic directly.

Add a `circuit_tests/` test when you change Circom circuits

Use this for:

- new circuit files,
- changed template parameters,
- changed witness expectations,
- changed circuit inputs.

Recommended file naming:

- either extend `circuit_tests/zkLogin.test.ts` when the new behavior is part of the existing zkLogin circuit family,
- or add a new `circuit_tests/<feature>.test.ts` file when the circuit area is distinct enough to deserve its own suite.

How To Update Existing Tests

Update tests according to the kind of production change you make.

If you change a handler response shape

Update:

- response field assertions,
- expected status codes,
- any shared helper assumptions in `tests_common_code/responseAssertions.ts`.

Example: if `handleGenerateProofEndpoint` returns a renamed field instead of `proofContent`, update `src/generateProofEndpoint.test.ts`.

If you change the proof generation workflow

Update:

- `src/generateProofEndpoint.test.ts`,
- `deployment/generateProof.test.ts`,
- any proof fixture logic in `tests_common_code/writeProofInFile.ts`.

That is important because both handler-level and deployment-level proof tests depend on the same proof file contract.

If you change canonical JWT or circuit input structure

Update:

- `tests_common_code/testDataForACompleteFlowOfZkLogin.ts`,
- the endpoint tests that call `circuitInputs()`,
- the circuit tests that reuse the merged session data.

This file acts as a central fixture source, so format changes ripple through multiple suites.

Practical Rules For New Tests

- Prefer direct handler testing over starting the HTTP server unless you specifically need route wiring coverage.
- Inject fake actions for expensive work such as proof generation or blockchain calls.
- Reuse fixture builders from `tests_common_code/` instead of copying long JWTs or circuit payloads into new tests.
- Keep filesystem writes inside dedicated test output directories such as `deployment/test_data`.

- Only use real proof generation when you intentionally want integration-level coverage; otherwise keep `REAL_PROOF` unset and use the stub path.